

Session 1: Introduction to Version Control

Topic: Fundamentals, Tool Ecosystem & Environment Setup

Core Commands: `git config` `git init` `git status`

1. Core Concepts

Version Control System (VCS): Software that tracks and manages changes to code or document files over time. It creates a complete history of revisions, allowing developers to review historical states, compare changes, and collaborate without overwriting work.

Centralized vs. Distributed Version Control

Feature	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Architecture	Single central server contains the complete file history.	Every client mirrors the full repository and history locally.
Examples	Subversion (SVN), Perforce (P4)	Git, Mercurial
Offline Work	Limited — requires network connection to commit or view logs.	Full capability — commits, branching, and logging work offline.
Risk Point	Single point of failure (if central server crashes, history is lost).	High redundancy — every clone acts as a full backup.

2. Overview of Tools & Platforms

It is important to distinguish between **Git** (the underlying CLI version control engine) and **Git Hosting Platforms** (web services providing cloud hosting, pull requests, and CI/CD pipelines).

Git The local command-line VCS engine developed by Linus Torvalds. Manages snapshots locally.	GitHub Cloud platform by Microsoft for hosting Git repos, code reviews, Actions CI/CD, and open-source collaboration.	GitLab All-in-one DevOps platform offering built-in CI/CD pipelines, container registry, and self-hosted server options.	Bitbucket Atlassian platform optimized for enterprise teams, featuring native integration with Jira and Trello.
---	---	--	---

3. Setup & Global Configuration

Installation Overview

- **macOS:** Install via Homebrew: `brew install git` (or Xcode Command Line Tools).
- **Windows:** Download and install Git for Windows (includes Git Bash terminal).
- **Linux (Ubuntu/Debian):** Install via apt package manager: `sudo apt update && sudo apt install git`

Configuring User Identity

Before creating commits, Git requires your identity information. This metadata is attached to every commit snapshot you produce.

```
# Set global username
git config --global user.name "Your Name"

# Set global user email (should match your GitHub/GitLab email)
git config --global user.email "your.email@example.com"

# Set default branch name (standard industry default: main)
git config --global init.defaultBranch main

# Verify configurations
git config --list
```

4. Hands-On Execution Walkthrough

STEP 1 Create Project Directory

Open terminal/command prompt and create a dedicated workspace folder.

```
# Create directory
mkdir intro-git-demo

# Navigate into directory
cd intro-git-demo
```

STEP 2 Initialize Git Repository

Transform the directory into a Git-tracked repository. This creates a hidden `.git` folder.

```
# Initialize local repository
git init
# Output: Initialized empty Git repository in ../../intro-git-demo/.git/
```

STEP 3 Inspect Repository Status

Check the state of working tree before creating any files.

```
# Check status
git status
# Output: On branch main, No commits yet, nothing to commit
```

5. Practice Exercise: Track a Simple Text File

Hands-On Exercise Tasks

- Create a simple text file named `notes.txt` inside `intro-git-demo/`.
- Add the text line: `Welcome to Version Control with Git!`
- Execute `git status` and observe that `notes.txt` appears under **Untracked files**.
- Run `git config --list` to confirm your `user.name` and `user.email` are correctly displayed.